

# ROOTで解析してみよう

---

サマーチャレンジ 2026 演習#10  
ミューオンの寿命測定と歳差運動  
九州大学 素粒子実験研究室  
文責：久戸瀬太一

# ROOTとは？

- ・ 欧州原子核研究機構（**CERN**）が開発した解析ソフトウェア・ライブラリ群
- ・ 独自のデータ構造を持ち、**多イベント・多変数**の処理に強い
- ・ 素粒子・原子核・宇宙（素核宇）など、高エネルギー分野での標準的な解析ツール

## 以下の環境で動作

- ・ Linux (WSL2でもOK)
- ・ macOS 14 Sonoma以降
- ・ Windows  
→ こだわりがなければWSL2を推奨

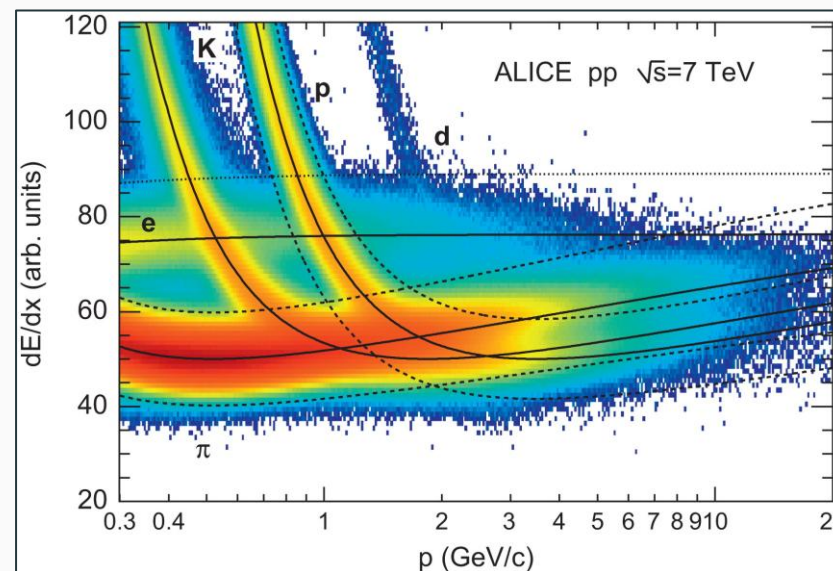
ここでは2通りの導入方法を紹介：

- ①パッケージマネージャを利用する
- ②ビルド済みバイナリをダウンロードする

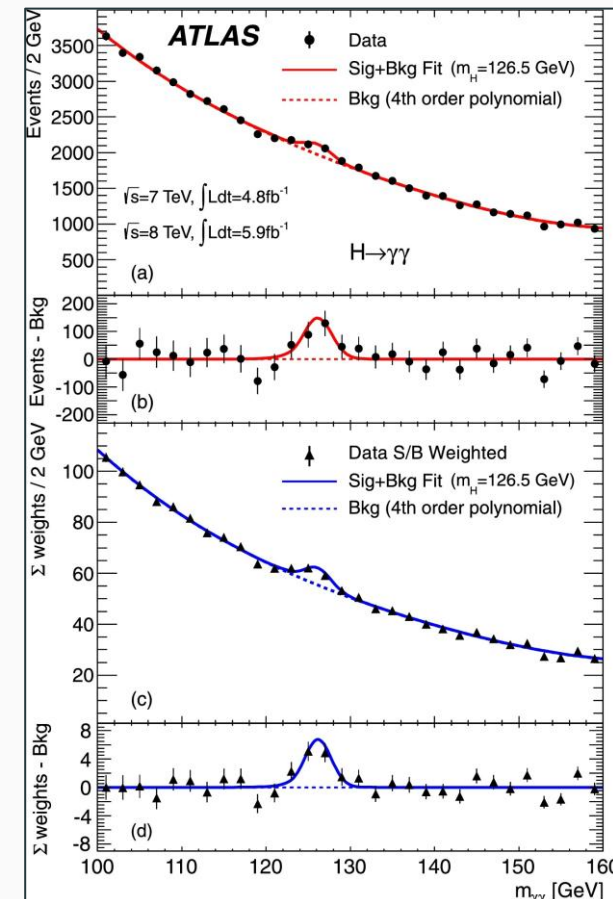
\*オープンソースソフトウェアなので、上級者はgit cloneしてもよい



## ROOTを使った解析例↓→



pp衝突で生じた荷電粒子を運動量、エネルギー損失についてプロットしたもの。実線は理論曲線、ヒートマップは各( $p$ ,  $dE/dx$ )で観測された荷電粒子数を表す。粒子ごとに異なる帯構造を作り、識別が可能である。



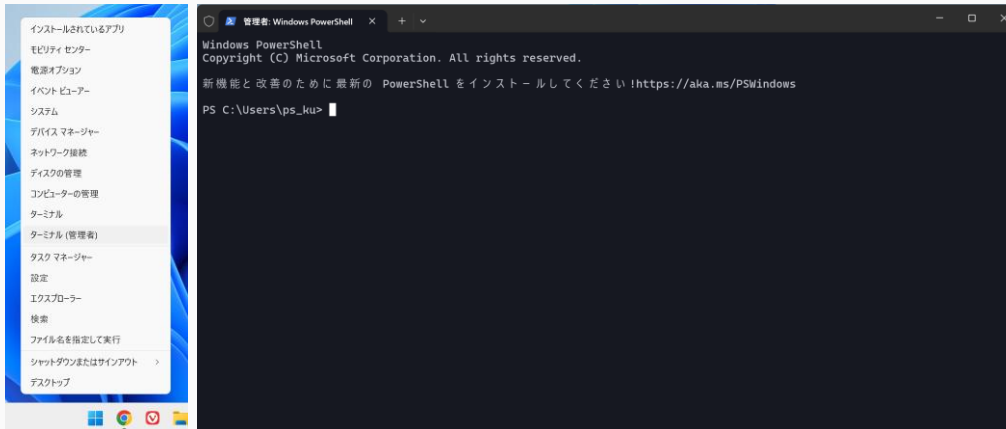
ATLAS実験による  $H \rightarrow \gamma\gamma$  候補事象の不変質量分布 [2]。126.5 GeV 付近に背景から有意な超過が見られる。Higgs粒子発見の決定打。

# 【準備】WSL2の導入（Windowsユーザ向け）

## Windows Subsystem for Linux 2

- ・ ・ ・ Windows上でLinux環境を動かす、仮想環境

### ① ターミナル（管理者）を起動する（を右クリック）



### ② `wsl --install --distribution AlmaLinux-10`を実行。

完了後、再起動する。

### ③ ユーザ名とパスワードを求められるので、設定する。

パスワードはこの後使うので忘れないこと。

### ④ 「`Ctrl` + `,`>プロファイル>(使用しているLinuxOS)>開始ディレクトリ」の「親プロセス ディレクトリの使用」からチェックを外し、`~`に設定・保存する。またターミナルで`cd ~`を実行する。

#### Tips：ディストリビューションの指定

`--distribution`以降を書かなければ、Ubuntuがインストールされる。世界シェアではUbuntuが多数を占めるが、自然科学分野ではCentOS（サポート終了）やAlmaLinuxが主流である。Ubuntuユーザ向けの手順も併記するので、どちらを選んでもよい。

#### Tips：④の解説

Linuxから見れば、Cドライブは外部のファイルシステムである。管理方法の違いから読み書きには時間がかかるので、作業はLinuxのホームディレクトリ「`~`」で行うことが推奨される。  
`cd`はchange directoryの略で、`cd ~`は「`~`に移動せよ」と読める。

# 【発展】 Visual Studio Codeを使ってみよう

VSCoDeを使えばコードの編集とターミナルの操作が1つのウィンドウで完結する。

## Linux / macOS / Windowsユーザー

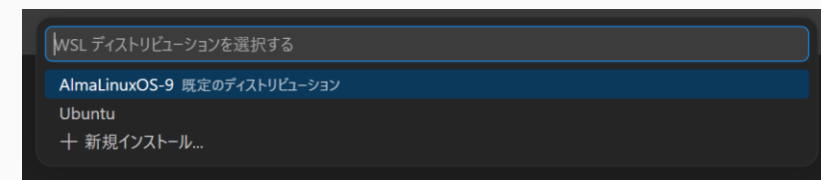
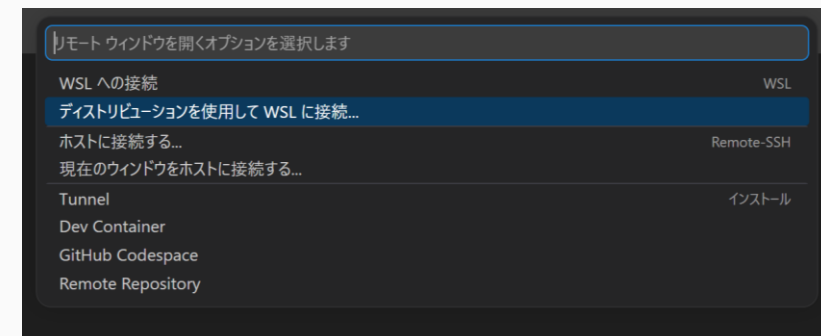
- ① VSCoDeを起動する。
- ② 「ターミナル>新しいターミナル」をクリック。  
ウィンドウ下部でターミナルが起動する。
- ③ `cd ~`を実行し、ホームディレクトリに移動する。



ターミナルの開き方

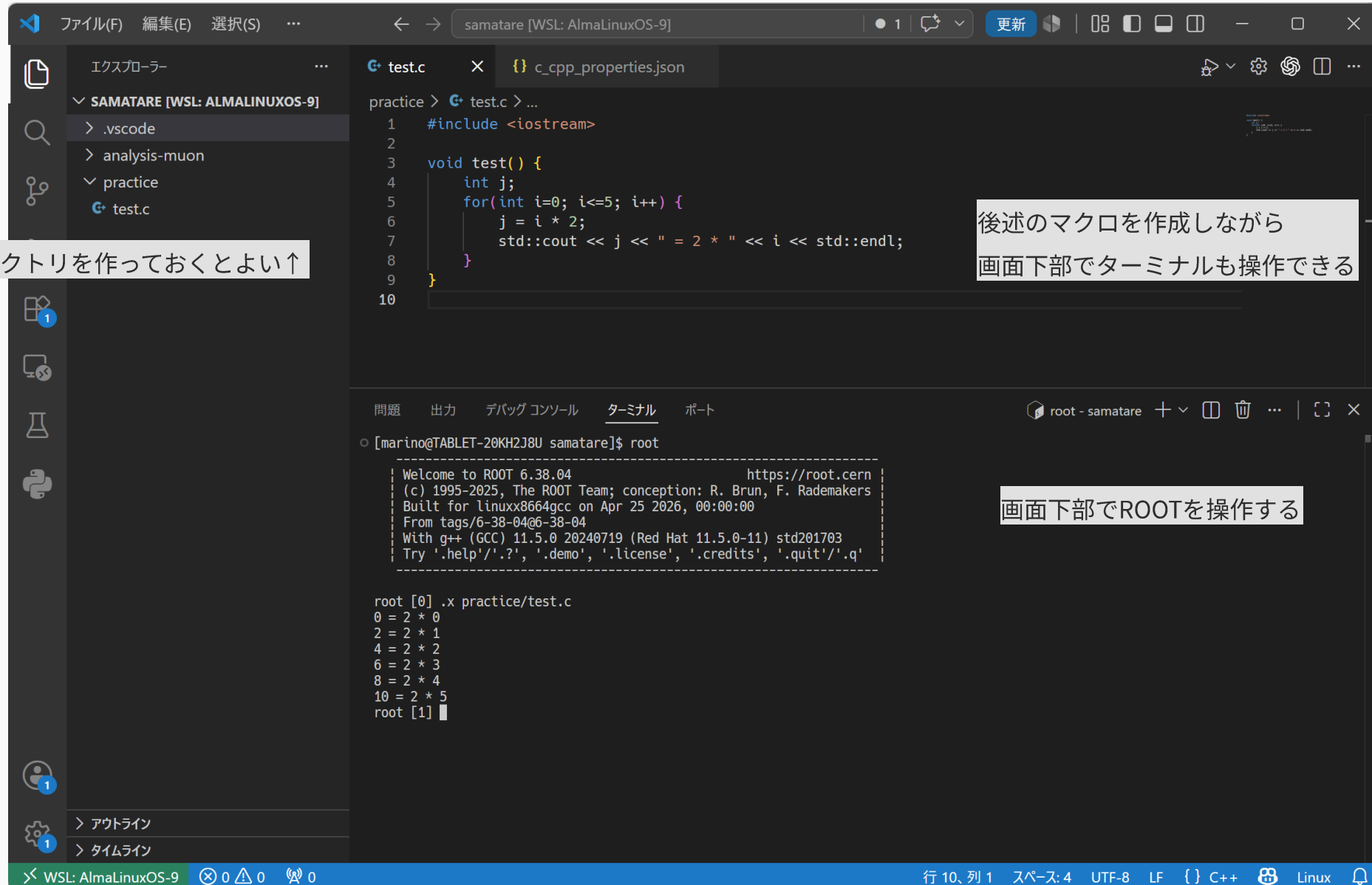
## WSL2ユーザー

- ① VSCoDeを起動する。
- ② ウィンドウ左下のをクリックし、  
「ディストリビューションを使用してWSLに接続...>AlmaLinux-10」を選択
- ③ 「ターミナル>新しいターミナル」をクリック。  
ウィンドウ下部でターミナルが起動する。
- ④ `cd ~`を実行し、ホームディレクトリに移動する。



WSL2の起動 in VSCode

# VSCodeを使った開発例



# パッケージマネージャ編：AlmaLinux / RHEL系, macOS, WSL2

ROOTコミュニティにより、パッケージが提供されている。

## ① 以下を実行する

\$は行頭記号なので入力しない

AlmaLinux / RHEL系

```
$ sudo dnf update
$ sudo dnf install epel-release
$ sudo dnf install root python3-root
```

macOS

```
$ brew install root
```

Is this ok [y/n]: はインストールの可否→yを入力する。

※エラーが出る！→FAQを参照。解決しなければビルド済みバイナリ編 for Alma...を参照

※Ubuntu / Debian系はパッケージが提供されていない。

→ビルド済みバイナリ編 for Ubuntuを参照のこと。

## ② rootを実行し、ROOTが起動することを確認する。

```
[marino@TABLET-20KH2J8U ~]$ root
-----
Welcome to ROOT 6.38.04                                     https://root.cern
(c) 1995-2025, The ROOT Team; conception: R. Brun, F. Rademakers
Built for linuxx86_64gcc on Apr 22 2026, 00:00:00
From tags/6-38-04@6-38-04
With g++ (GCC) 14.3.1 20251022 (Red Hat 14.3.1-4) std201703
Try '.help'/'?', '.demo', '.license', '.credits', '.quit'/'.'
-----
root [0] █
```

※起動したけどなんか変

→FAQを参照。

.qで終了する。早くできた人は、.demoで機能を試してみるとよい。

### Tips：パッケージマネージャとは

プログラムやメタデータなどをひとまとめにしたデータ群をパッケージという。OSにはパッケージの一元管理システムであるパッケージマネージャが搭載されている。

### Tips：①の解説

パッケージのインストールはコンピュータに変更を加えるので、管理者の承認が必要。sudoはsuperuser doの略で、管理者権限で実行することを意味する。

dnf / apt / Homebrewは各OSに搭載されたパッケージマネージャ。つまり左のコマンドは「管理者が命令する。dnfを使ってepel-releaseをinstallせよ。」と読める。

### Tips：古いLinuxコンピュータを使っている方へ

古いRHEL系ではdnfの代わりにyumを使うことがある。研究室の古いコンピュータではCentOSやScientific Linuxが搭載されていることがあり、これらはyumを使う。AlmaLinuxでも利用可能だが、非推奨。

# ビルド済みバイナリ編 for AlmaLinux / RHEL

- ① 前提パッケージを導入する。必要なパッケージは公式サイトに掲載されている：[Dependencies – ROOT](#)

AlmaLinux / RHEL系

```
$ sudo dnf update
$ sudo dnf install -y git make cmake gcc-c++ gcc binutils libX11-devel libXpm-devel
libXft-devel libXext-devel python openssl-devel xrootd-client-devel xrootd-libs-devel
```

- ② ホームディレクトリに opt/ を作り、移動。

[公式サイト](#)で自分の環境向けのファイル名を確認し、ダウンロード・展開する。

AlmaLinux / RHEL系

```
$ mkdir ~/opt
$ cd ~/opt |→これより後ろは自分のOSに合ったものを公式サイトで確認する
$ wget https://root.cern/download/root_v6.40.02.Linux-almalinux10.2-x86_64-gcc14.3.tar.gz
$ tar -xvf root_v6.40.02.Linux-ubuntu24.04-x86_64-gcc13.3.tar.gz
```

- ③ PATHを通し、起動確認。

AlmaLinux / RHEL系

```
$ echo 'source ~/opt/root/bin/thisroot.sh' >> ~/.bashrc
$ source ~/.bashrc
$ root
```

.qで終了する。早くできた人は.demoで機能を試してみるとよい。

※起動したけどなんか変 / 起動しない→FAQを参照

Tips : Linuxコマンドを覚えよう

mkdir(make directory): ディレクトリ (フォルダ) を作る

wget(www get): ウェブからファイルを取得する

tar(tape archive): tar形式のファイル进行处理する。tar.gzはtar/gzip形式の圧縮ファイルで、xvfはextract (展開) , verbose (ログ表示) , file (ファイルを指定)

# ビルド済みバイナリ編 for Ubuntu

- ① 前提パッケージを導入する。必要なパッケージは公式サイトに掲載されている：[Dependencies – ROOT](#)

Ubuntu / Debian系

```
$ sudo apt update
$ sudo apt install -y binutils cmake dpkg-dev g++ gcc libssl-dev git libx11-dev
libxext-dev libxft-dev libxpm-dev python3 libtbb-dev libvdt-dev libgif-dev
```

- ② ホームディレクトリに opt/ を作り、移動。Ubuntuのバージョンはlsb\_release -aで確認できる。  
[公式サイト](#)で自分の環境向けのファイル名を確認し、ダウンロード・展開する。

Ubuntu / Debian系

```
$ mkdir ~/opt
$ cd ~/opt |→これより後ろは自分のOSに合ったものを公式サイトで確認する
$ wget https://root.cern/download/root_v6.40.02.Linux-ubuntu24.04-x86_64-gcc13.3.tar.gz
$ tar -xvf root_v6.40.02.Linux-ubuntu24.04-x86_64-gcc13.3.tar.gz
```

- ③ PATHを通し、起動確認。

Ubuntu / Debian系

```
$ echo 'source ~/opt/root/bin/thisroot.sh' >> ~/.bashrc
$ source ~/.bashrc
$ root
```

.qで終了する。早くできた人はdemoで機能を試してみるとよい。

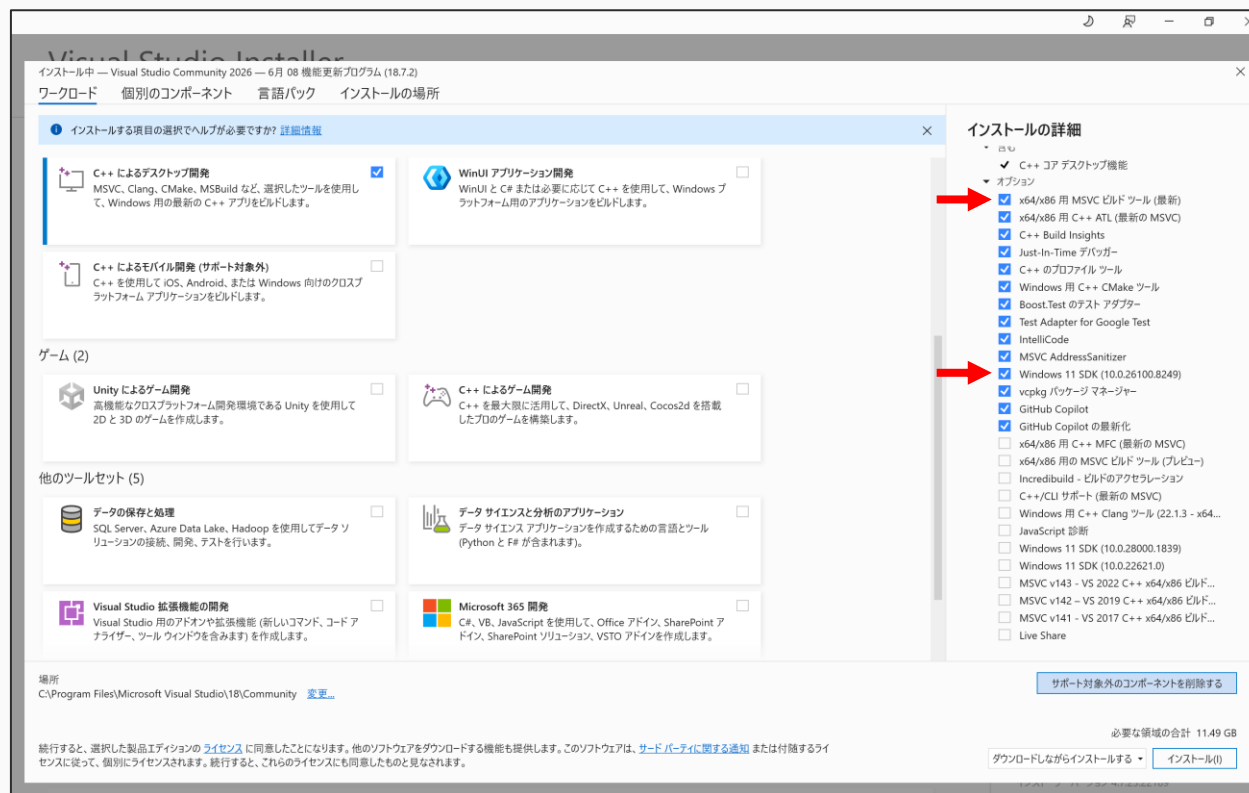
※起動したけどなんか変 / 起動しない→FAQを参照

## Tips : Tabを活用しよう

これらの入力を行うのは手間ではないだろうか？  
ターミナルでファイル名を入力する時、Tabキーを押せば補完・候補を表示できる。tar -xvf rooまで入力してTabを押してみよ。補完できなければファイルが存在しないか、そこまで誤字をしている。

# ビルド済みバイナリ編 for Windows

- ① [Visual Studio 2026](#)インストーラをダウンロードする。
- ② インストーラを起動し、「Visual Studio Community > C++によるデスクトップ開発」を選択。
- ③ 右のチェックボックスのうち、以下をチェックし、インストール。
  - x64/x86用MSVCビルドツール（最新）
  - Windows 11 SDK

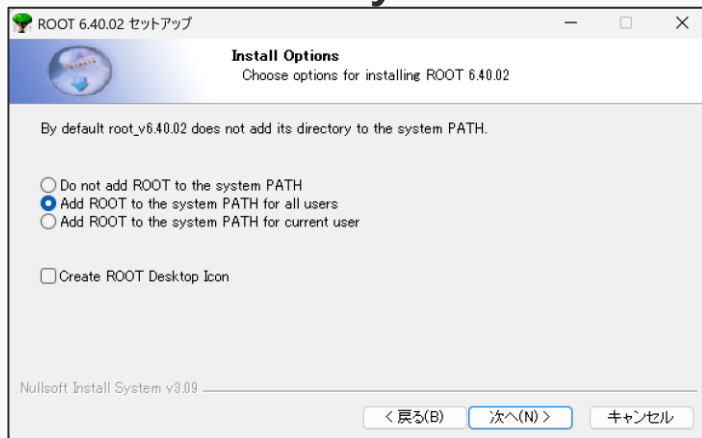


今後C++で開発をしたい人はデフォルトのままでもよい。また、後から必要に迫られたら、再インストールすればよい。

# ビルド済みバイナリ編 for Windows

④ [ROOTインストーラ](#)をダウンロードし、実行。⑤⑥に注意し進める。

⑤ **Add ROOT to the system PATH for all users**を選択 ※うっかり飛ばしてしまった→FAQ「起動できない」を参照



⑥ **Full Installation**を選択 (デフォルトのまま)



|                                                                            |                                                                       |      |
|----------------------------------------------------------------------------|-----------------------------------------------------------------------|------|
| Windows Visual Studio 2022 32-bit x86                                      | <a href="#">root_v6.40.02.win32.python311.vc17.exe</a>                | 132M |
| Windows Visual Studio 2022 32-bit x86                                      | <a href="#">root_v6.40.02.win32.python311.vc17.zip</a>                | 181M |
| Windows Visual Studio 2025 32-bit x86 (release with debugging information) | <a href="#">root_v6.40.02.win32.python314.vc18.relwithdebinfo.exe</a> | 438M |
| Windows Visual Studio 2025 32-bit x86 (release with debugging information) | <a href="#">root_v6.40.02.win32.python314.vc18.relwithdebinfo.zip</a> | 696M |
| Windows Visual Studio 2022 64-bit x64                                      | <a href="#">root_v6.40.02.win64.python311.vc17.exe</a>                | 137M |
| Windows Visual Studio 2022 64-bit x64                                      | ✗ <a href="#">root_v6.40.02.win64.python311.vc17.zip</a>              | 188M |
| Windows Visual Studio 2025 64-bit x64 (release with debugging information) | <a href="#">root_v6.40.02.win64.python314.vc18.relwithdebinfo.exe</a> | 447M |
| Windows Visual Studio 2025 64-bit x64 (release with debugging information) | <a href="#">root_v6.40.02.win64.python314.vc18.relwithdebinfo.zip</a> | 709M |

普通はWindows Visual Studio 2022 64-bit x64を選択。一部、古いPCを使用している場合は、CPUのアーキテクチャがx32の場合がある。不安であれば「設定>システム>バージョン情報>デバイスの仕様」で、システムの種類にx64 ベース プロセッサと書かれていることを確認する。x32であれば32-bit x86用のインストーラをダウンロードする。ARMの方（Snapdragon Xなど）→FAQを参照

⑦ rootを実行し、起動確認。

.qで終了する。早くできた人は、demoで機能を試してみるとよい。  
※起動したけどなんか変 / 起動しない→FAQを参照

# ビルド済みバイナリ編 for macOS

- ① CPUのアーキテクチャとOSを確認する（Intel Mac、Appleシリコンなど聞いたことがあるだろう）

「 > このMacについて」からプロセッサ（チップ）とOSを確認。

- ② [公式サイト](#)から自分の環境にあったインストーラをダウンロード。

|                            |                                                                                                                                                  |      |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|------|
| macOS 14.8 x86_64 Xcode 16 | <a href="#">root_v6.40.02.macos-14.8-x86_64-clang160.pkg</a>                                                                                     | 477M |
| macOS 14.8 x86_64 Xcode 16 | <a href="#">root_v6.40.02.macos-14.8-x86_64-clang160.tar.gz</a>                                                                                  | 312M |
| macOS 15.7 arm64 Xcode 17  | <a href="#">root_v6.40.02.macos-15.7-arm64-clang170.pkg</a>                                                                                      | 463M |
| macOS 15.7 arm64 Xcode 17  | <a href="#">root_v6.40.02.macos-15.7-arm64-clang170.tar.gz</a>                                                                                   | 308M |
| macOS 26.5 arm64 Xcode 21  | <a href="#">root_v6.40.02.macos-26.5-arm64-clang210.pkg</a>                                                                                      | 479M |
| macOS 26.5 arm64 Xcode 21  |  <a href="#">root_v6.40.02.macos-26.5-arm64-clang210.tar.gz</a> | 320M |

- ③ 一度起動する。ブロックされるので、

「設定>プライバシーとセキュリティ>セキュリティ」から「このまま開く」

- ④ インストーラの指示に従い、インストールする。

- ⑤ 完了後、ターミナルで`root`を実行して、起動確認。

`.q`で終了する。早くできた人は`.demo`で機能を試してみるとよい。



# C / C++の書き方：変数と関数の型

C言語はPythonと異なり、変数にどのようなもの（型）を代入するか、あらかじめ予告（宣言）する。

```
int i = 46;           // 「iは整数(integer)です。46を代入します」
double pi = 3.14      // 「piは倍(double)精度浮動小数点数です。3.14を代入します」
```

数学同様、関数は値（戻り値）を返す： $f(x) = x^2$  という関数は  $x = 2$  を渡すと  $f(2) = 4$  を返す。

```
// 2乗を計算する関数を定義し、計算結果を表示しよう
double f(double x) {           // 「小数を扱う関数f(x)を定義する。xは小数を扱う変数である」
    double y = x * x;          // 「小数を扱うyを定義し、x*xを代入する」
    return y;                  // まだ計算しただけ。「結果を返す(return)」
}

int main() {
    double i = 46.2;
    double j;
    j = f(i);                  // f(x)は計算結果をreturnするので、受け取る
    cout << "square of " << i << " is " << j << endl;      // 表示する
    return 0;                  // 右のTips参照
}
```

## Tips：main関数の型

mainは実行の本体であり、実行すれば戻り値はいらないのに、なぜvoidではなくintなのだろうか。①昔はvoid型がなかったので、形式的にreturn 0を書いてint型ということにした ②0は真偽判定で偽を意味するので、実行中ですか→いいえ、の判定に使える。

もし関数が表示までしてくれるなら？

```
void g(double x) {
    double y = x * x;
    cout << "square of " << x << " is " << y << endl;
}
```

main関数で受け取る必要がなくなる。受け取らないのでreturnもない。つまり戻り値はいらない→void型

# ROOTとは？ season 2

ROOTはC++を使った解析フレームワークである。

例：  

```
root[0]int i = 46;
root[1]cout << "i = " << i << endl;
```

複数行の入力も可能：

```
root[0] {
root (cont'ed, cancel with .@)[1]int j;
root (cont'ed, cancel with .@)[2]for(int i = 1; i <= 5; i++) {
root (cont'ed, cancel with .@)[3]j = i * 2;
root (cont'ed, cancel with .@)[4]cout << i << " times 2 is " << j << endl;
root (cont'ed, cancel with .@)[5]}
root (cont'ed, cancel with .@)[6]}
```

マクロを作成することも可能：

```
test.C
void test() {
    int j;
    for(int i=1; i<=5; i++) {
        j = i * 2;
        cout << i << " * 2 = " << j << endl;
    }
}
```

## Tips : C++の説明

int iは「iは整数を代入する変数である」という宣言。ここでは46を代入した。coutはconsole(ターミナル)output(出力)の略で、イメージとしては出力したい内容を<<で画面に押し込んでいる。endlは改行を意味する(end line)。Cを使ったことがあれば、printf()を使ってもよい。

拡張子を大文字の.CにしてROOTマクロであることを明示する

```
root[0].x test.C
```

~/SC26/のようなサマチャレ用のディレクトリを作り、練習用のpractice/を作成してマクロを保存しよう。(ヒント：7ページ / VSCodeならGUI操作可)

上2例のような対話型では、入力を間違えても訂正できない。VSCodeを使えば、このようにマクロを作成しながらまとめて実行できる。

勿論、EmacsやVim、nanoを使ってもよい。

# ROOTファイルを開く

抵抗値を1000回繰り返し測定して、example1.rootを取得した。

```
1. $ cd ~/SC26/practice
$ root -l
root[0]TFile *f = TFile::Open("example1.root");
```

TFileはROOTの標準I/Oクラス。 [ROOT: TFile Class Reference](#)

Open関数は.rootファイルからTFileオブジェクトを作成する。

戻り値はTFile\*型なので、変数はポインタとして宣言する。

```
2. root[1]f->ls();
TFile**      example1.root
TFile*       example1.root
KEY: TTree   tree;1  Event list from SiPM
```

中身を確認する。

ls()はTFileクラスのメンバ関数で、Open関数で取得したTFileオブジェクトの中身を表示する。

TTreeはイベントデータを保存するROOTのクラス。

測定データが、TTree型のtreeという名前で保存されていることが分かった。

## Tips : C++の説明

クラスとは関数を格納したツールボックスのようなもの。TFile::OpenはTFileクラスで定義されている（メンバ関数という）Open関数を呼び出している。

ポインタとはメモリの住所。メモリに保存された内容（インスタンス）ではなく、そのメモリの住所に対して処理を実行するのが、C++の特徴である。アロー演算子->はポインタが指すインスタンスに対し、メンバ関数を実行する。

# ROOTファイルを開く

```
3. root[2] TTree* tree = nullptr;  
   root[3] f->GetObject("tree", tree);  
   root[4] tree->Print();  
   root[5] tree->Scan();
```

保存されているデータを取得する。

root[2] 測定データを格納する変数を宣言し、初期化する。(ls())でTTree型と分かっている)

root[3] TFile::GetObject関数は指定の名前のオブジェクトを探し、変数に格納する関数。

root[4] TTree::Print()はツリーの概要を表示する関数。

root[5] TTree::Scan()は具体的なデータをリストで表示する関数。

抵抗値resistanceが記録されていることが確認できただろうか？

```
4. root[6] tree->Draw("resistance>>hist_R(200,999,1001)");
```

ヒストグラムを表示する。

TTree::Draw()は引数で設定したようにヒストグラムを作成・表示する関数。

treeの中からresistanceというブランチを探し、hist\_Rというヒストグラムを作成する。

ビンの数は200、横軸は999から1001。

## Tips：横軸の範囲

データ範囲を知らなければ横軸は設定しづらい。Scan()の出力をすべて見るのは大変。

TTree::GetMinimum()、TTree::GetMaximum()で最小値・最大値を取得できる。戻り値はdouble型。

```
$ double rmin = tree->GetMinimum("resistance");  
$ cout << rmin << endl;
```

# ROOTファイルを開く

5. 測定データは何らかの理論によって与えられ、無限回の試行で、ある理論曲線を描く。

問：測定系の誤差により、測定した抵抗値は真の値の周りで統計的に揺らいでいる。どのような統計的分布か？

```
root[7]TH1* hist_R = (TH1*)gDirectory->Get("hist_R");
```

フィットしたいヒストグラムを取得する。

4で作成したヒストグラムhist\_RはgDirectoryというメモリに格納されている。これを取り出す。

TObject::Get()は引数で指定したオブジェクトを取り出す関数。戻り値はTObject\*という汎用的な型。

ここでは1次元ヒストグラムを扱うので、TH1\*型に変換し、hist\_Rに格納する。

```
root[8]TF1* fit = new TF1("fit", "[0]*exp(-0.5*pow((x-[1])/[2], 2))", 999, 1001);  
root[9]fit->SetParameter(0, 20);  
root[10]fit->SetParameter(1, 1000);  
root[11]fit->SetParameter(2, 0.2);
```

フィットする関数を定義する。

TF1は一次元関数を扱うクラス。

TF1::TF1()は引数に応じた関数を定義する関数。引数で関数名、関数、下限と上限を設定。

TF1::SetParameter()はパラメタの初期値を定義する関数。

# ROOTファイルを開く

5. `root[12]hist_R->Fit("fit", "RL");`

フィッティングを実行する。

TH1::Fit()は引数に応じてフィッティングする関数。

フィッティングに使う関数は先ほど定義したfit。

"R"は先ほど定義した範囲内をフィッティングする、という指示 (Range)

"L"はビンがポアソン分布に従うと仮定する、という指示 (Log likelihood)

フィッティング曲線と最尤推定値が確認できただろうか？

# 解析手順のまとめ

|   |                                 |                                                                                                                               |
|---|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| 1 | rootファイルを開く<br>中身を確認する          | <pre>TFile *f = TFile::Open("ファイルパス");<br/>f-&gt;ls();</pre>                                                                  |
| 2 | データを入れる変数を作って初期化<br>データ（ツリー）の取得 | <pre>TTree *tree = nullptr;<br/>f-&gt;GetObject("取得したいツリー", tree);</pre>                                                      |
| 3 | データの確認                          | <pre>tree-&gt;Print();<br/>tree-&gt;Scan();</pre>                                                                             |
| 4 | ヒストグラムの作成・表示<br>取得              | <pre>tree-&gt;Draw("データ名&gt;&gt;ヒストグラム名(分割数, min, max)");<br/>TH1* hist = (TH1*)gDirectory-&gt;Get("ヒストグラム名");</pre>          |
| 5 | フィッティング関数を定義<br>フィッティングを実行      | <pre>TF1 func = ("func", "関数形([i])", min, max);<br/>func-&gt;SetParameter(i, init);<br/>hist-&gt;Fit("func", "option");</pre> |

# マクロの例

example1.C

```
#include "TFile.h"
#include "TTree.h"
#include "TH1.h"
#include "TF1.h"

void example1(){
    TFile *f = TFile::Open("~/SC26/practice/resistance.root");
    TTree *tree = nullptr;
    f->GetObject("tree", tree);
    tree->Draw("resistance>>hist_R(200, 999, 1001)");
    TH1* hist_R = (TH1*)gDirectory->Get("hist_R");
    TF1* fit = new TF1("fit", [0]*exp(-0.5*pow((x-[1])/[2],2)), 999, 1001);
    fit->SetParameter(0, 20);
    fit->SetParameter(1, 1000);
    fit->SetParameter(2, 0.2);
    hist_R->Fit("fit", "RL");
}
```

ライブラリヘッダをincludeしなくてもインタプリタは理解できるが、マクロの場合は明示する方が丁寧。  
f, tree, hist\_R, fitは予約語でなければわかりやすい変数名を自由に設定してよい。

# 演習1：宇宙線ミュオンの天頂角分布

問：宇宙線ミュオンは大気を通過して地表に到達するため、真上からのミュオンが最も多く、横方向からのミュオンは少ない。天頂角  $\theta$  [rad] で到達する数は、 $A \cos^b c\theta$  で書けることが分かっている。Example2.rootは、宇宙線ミュオンが観測されたときの天頂角を記録したものである。縦軸をカウント、横軸を角度とするヒストグラムを作成し、フィットするマクロを作成せよ。

ヒント：問題設定と分布が異なるだけで、やることは同じ。

発展：Appendix Aを見て、次の関数を用いてみよ。

- `TFile::IsZombie()`を用いて例外処理を行う。
- `TF1::SetParameters()`で初期値をまとめて設定する。

# 演習1：解答例

まずファイルの中身を確認しておく。

```
root[0]TFile *file = TFile::Open("example2.root");
root[1]file->ls();
root[2]TTree *tree = nullptr;
root[3]file->GetObject("tree", tree);
root[4]tree->Print();
root[5]tree->Scan();
```

マクロを作成する

```
#include "TFile.h"
#include "TTree.h"

void example2(){
    TFile *file = TFile::Open("~/SC26/practice/example2.root");
    if(!file || file->isZombie()) return; //大型の解析では例外処理が必須。ここでは任意。
    TTree *tree = nullptr;
    file->GetObject("tree", tree);
    tree->Draw("zenith_angle>>hist_theta(500, 0, 90)");
    TH1* hist_theta = (TH1*)gDirectory->Get("hist_theta");
    TF1* fit_func = new TF1("fit_func", "[0]*pow(cos([1]*x),[2])", 0, 90);
    fit_func->SetParameters(1000, 1, 2);
    hist_theta->Fit("fit_func", "RL");
}
```

# Appendix A: ROOT API 早見表

## TFileクラス

- `TFile* TFile::Open(const char* url);`
- `TObject* TDirectoryFile::Get(const char* namecycle);`
- `void TDirectoryFile::ls("Option_t* option = "");`

## TTreeクラス

- `Long64_t TTree::Draw(const char* varexp);`

## TH1クラス

- `TH1::TH1(const char* name, const char* title, Int_t nbinsx, Double_t xlow, Double_t xup);`
- `TH1::Fit(const char* name, Option_t option = "");`

## TF1クラス

- `TF1::TF1(const char* name, const char* formula, Double_t xmin, Double_t xmax);`
- `void TF1::SetParameters(Double_t p0, Double_t p1, Double_t p2);`

- CentOSでもできるか？

⇒ できるが、すでにサポートが終了している。

- パッケージが見つからない： `Error: Unable to find a match: ...` `E: Failed to fetch http://...`

⇒ まずはパッケージマネージャが古いことを疑う・・・ `sudo dnf update` / `sudo apt update`

それでもダメなら[データベース](#)でパッケージ名を検索し、自分のOSのページからPackage nameを探す。

例えばAlmaLinux 10ではlibXft-develだが、Ubuntuではlibxft-dev。OSやバージョンによって名前が変わることがある。公式サイトやスライドの情報が古い場合があるので、このように検索されたい。

- PATHを通すとは？ `echo 'source ~/opt/root/bin/thisroot.sh' >> ~/.bashrc` は何をしている？

⇒ thisroot.shはROOTを起動するためのスクリプト。これをコンピュータに認識させることで、`root`と打てばROOTが起動するようになる。これを「PATHを通す」という。

公式サイトではPATHの通し方； `source ...`しか書かれていないが、これではターミナルを開くたびにPATHを通さなければならない。`echo ...`はこれを.bashrcという、自動実行される設定ファイルに追記して、明日以降の手間を省いてる。

Windowsではこの通りにできないので、次頁の「起動できない」を参照。

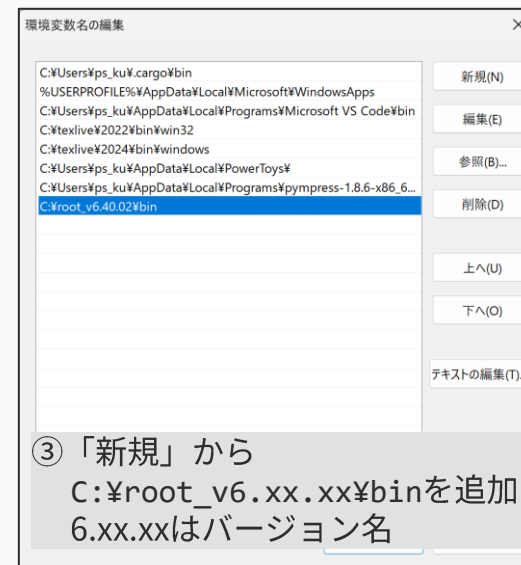
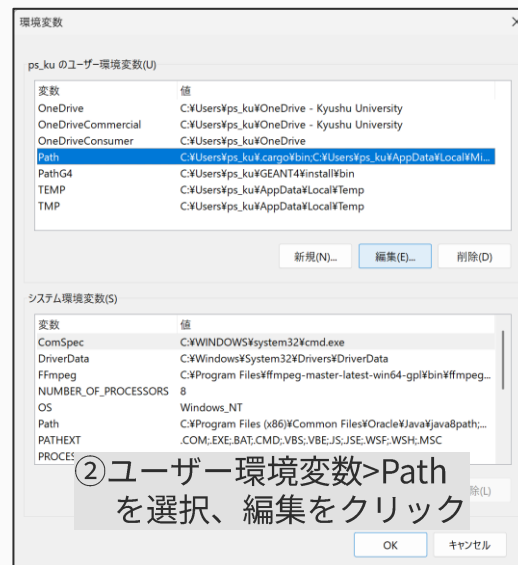
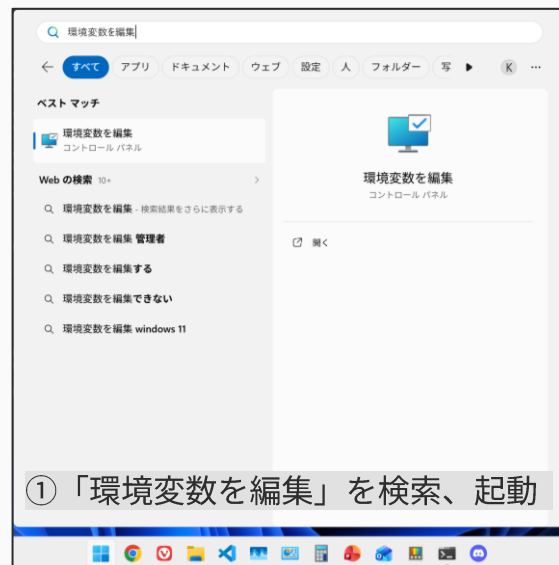
- 起動できない : `-bash: root: command not found` `root: command not found` (AlmaLinux / Ubuntu)

‘root’ は、内部コマンドまたは外部コマンド、  
操作可能なプログラムまたはバッチ ファイルとして認識されていません。 (Windows)

⇒ ビルド済みバイナリの場合、PATHが通っていない。

Linux: `echo 'source ~/opt/root/bin/thisroot.sh' >> ~/.bashrc`を実行。

Windows: 環境変数を編集する。



- 起動できるが右のようなエラーが出る

⇒ ROOT/ClingがC++標準ライブラリヘッダを見つけられていない。

C++のコンパイラが存在しない可能性が高い。

Linux : `sudo dnf install gcc-c++ gcc / sudo apt install g++ gcc`

Windows : Visual C++をインストールする。

Visual Studio 2026インストーラをダウンロード・起動し、

「Visual Studio Community 2026 > C++によるデスクトップ開発」

を選択。以下をインストールする。

- x64/x86用MSVCビルドツール（最新）
- Windows 11 SDK

```
PS C:\Users\ps_ku> root
RegOpenKeyEx(HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Microsoft SDKs\Windows
RegQueryValueEx(KitsRoot10): returned 2:
warning: unable to find a Visual Studio installation; try running Clang
input_line_1:1:10: fatal error: 'new' file not found
#include <new>
^~~~~~
input_line_3:51:10: fatal error: 'string' file not found
#include <string>
^~~~~~
-----
| Welcome to ROOT 6.40.02                                     https://root.cern |
| (c) 1995-2025, The ROOT Team; conception: R. Brun, F. Rademakers |
| Built for win64 on Jun 10 2026, 10:31:50 |
| From tags/v6-40-02@v6-40-02 |
| With MSVC 19.39.33521.0 std201703 |
| Try '.help'/'?', '.demo', '.license', '.credits', '.quit'/'q' |
-----
input_line_7:1:10: fatal error: 'iostream' file not found
#include <iostream>
^~~~~~
root [0]
```

- **CPUがARM系なのだが？**

⇒ Linux / WSL2なら問題ない。macOSかつビルド済みバイナリを使うなら、ARM版のインストーラを選択する。

Windowsの場合、ARM版のインストーラは用意されていない。ただしWindows on Armにはx86/x64アプリのエミュレーション機能があり、Windows10なら32-bit x86、Windows11なら64-bit x64が使えるかもしれない。インストールし、動作に不具合があればWSL2を利用する。（筆者の手元にARM機がないため検証不可。また、インストールされているPythonがARMネイティブであればPyROOTは使えない）

- **Ubuntuのバージョンに対応しているReleaseがない。**

⇒ Ubuntuのバージョン名はリリース日のYY.MMになっている（例：Ubuntu 23.04→23年4月リリース）  
Ubuntuのリリース日以降の[ROOT Release](#)をまず探す。  
なければ一つ前のバージョン用を試してみる。（例：Ubuntu 23.04→22.04用を試す）